

Dataset Analysis for Decision-Making

27/03/2025

HIV_Epidemiology... .csv
19.2 MB

Analyze the provided dataset to identify its key characteristics for data-driven decision-making. Upon receiving a specific analytical question, address it strictly using the dataset. Your deliverables must include a clear, concise answer to the question and the Python code that extracts and processes the data used to derive your answer.

qwen-max-latest

To analyze the dataset and address your request, I will first load the dataset to examine its structure and contents. This will allow me to identify key characteristics such as column names, data types, missing values, and general trends. Once the dataset's structure is understood, I can proceed to answer specific analytical questions using Python code.

Let's start by loading and inspecting the dataset.

Step 1: Load and Inspect the Dataset

Below is the Python code to load the dataset and display its key characteristics:

```
python
1 import pandas as pd
2
3 # Load the dataset
4 file_path = 'HIV_Epidemiology_Children_Adolescents_2024_out.csv'
5 data = pd.read_csv(file_path)
6
7 # Display basic information about the dataset
8 print("Dataset Overview:")
9 print(data.info())
10
11 # Display the first few rows of the dataset
12 print("\nFirst 5 Rows of the Dataset:")
13 print(data.head())
14
15 # Check for missing values
16 print("\nMissing Values per Column:")
17 print(data.isnull().sum())
18
19 # Summary statistics for numerical columns
20 print("\nSummary Statistics:")
21 print(data.describe())
```

Step 2: Key Characteristics of the Dataset

After running the above code, I will summarize the key characteristics of the dataset, such as:

- The number of rows and columns.
- Column names and their data types.
- Presence of missing or null values.
- Descriptive statistics for numerical columns (e.g., mean, median, min, max).
- Any notable patterns or trends in the data.

Step 3: Address Specific Analytical Questions

Once you provide a specific analytical question, I will write Python code to extract and process the relevant data from the dataset to answer the question. The code will include comments explaining each step.

Please provide the specific analytical question you'd like answered, and I will proceed with the analysis.

Compare ARIMA and an LSTM-based deep learning model for predicting HIV incidence among adolescents (15–19) in the Americas (2024–2026). Evaluate performance using RMSE.

qwen-max-latest

To compare the ARIMA and LSTM-based deep learning models for predicting HIV incidence among adolescents (15–19) in the Americas from 2024 to 2026, we need to follow these steps:

Step 1: Load and Prepare the Dataset

We will extract the relevant data for adolescents (15–19) in the Americas. This involves filtering the dataset based on the age group and geographic region.

Step 2: Preprocess the Data

For both models:

- **ARIMA** : Requires a univariate time series of HIV incidence.
- **LSTM** : Requires scaling of the data and creation of sequences for training.

Step 3: Train and Evaluate Models

- **ARIMA** : Fit the model using historical HIV incidence data and forecast for 2024–2026.
- **LSTM** : Train the LSTM model on historical data, validate it, and forecast for 2024–2026.
- Compute the **Root Mean Squared Error (RMSE)** for both models using a validation/test set.

Step 4: Compare Performance



Compare the RMSE values of ARIMA and LSTM to determine which model performs better for this specific prediction task.

Below is the Python code to implement this comparison:

```
python
1 import pandas as pd
2 import numpy as np
3 from statsmodels.tsa.arima.model import ARIMA
4 from sklearn.preprocessing import MinMaxScaler
5 from sklearn.metrics import mean_squared_error
6 from tensorflow.keras.models import Sequential
7 from tensorflow.keras.layers import LSTM, Dense
8 import math
9
10 # Step 1: Load and Filter the Dataset
11 file_path = 'HIV_Epidemiology_Children_Adolescents_2024_out.csv'
12 data = pd.read_csv(file_path)
13
14 # Filter for adolescents (15-19) in the Americas
15 adolescents_americas = data[(data['Age_Group'] == '15-19') & (data['Region'] == 'Americas')]
16
17 # Extract the time series of HIV incidence (assuming column name is 'HIV_Incidence')
18 time_series = adolescents_americas['HIV_Incidence'].values
19
20 # Step 2: Preprocess Data for ARIMA and LSTM
21 train_size = int(len(time_series) * 0.8)
22 train, test = time_series[:train_size], time_series[train_size:]
23
24 # For LSTM: Scale the data and create sequences
25 scaler = MinMaxScaler(feature_range=(0, 1))
26 train_scaled = scaler.fit_transform(train.reshape(-1, 1))
27 test_scaled = scaler.transform(test.reshape(-1, 1))
28
29 def create_sequences(data, seq_length):
30     X, y = [], []
31     for i in range(len(data) - seq_length):
32         X.append(data[i:i+seq_length])
33         y.append(data[i+seq_length])
34     return np.array(X), np.array(y)
35
36 seq_length = 10 # Example sequence length
37 X_train, y_train = create_sequences(train_scaled, seq_length)
38 X_test, y_test = create_sequences(test_scaled, seq_length)
39
40 # Step 3: Train ARIMA Model
41 arima_model = ARIMA(train, order=(5, 1, 0)) # Example order (p, d, q)
42 arima_fit = arima_model.fit()
43 arima_forecast = arima_fit.forecast(steps=len(test))
44
45 # Calculate RMSE for ARIMA
46 arima_rmse = math.sqrt(mean_squared_error(test, arima_forecast))
47 print(f"ARIMA RMSE: {arima_rmse}")
48
49 # Step 4: Train LSTM Model
50 lstm_model = Sequential([
51     LSTM(50, activation='relu', input_shape=(seq_length, 1)),
52     Dense(1)
53 ])
54 lstm_model.compile(optimizer='adam', loss='mean_squared_error')
55 lstm_model.fit(X_train, y_train, epochs=50, batch_size=32, verbose=0)
56
57 # Predict with LSTM
58 lstm_predictions = lstm_model.predict(X_test)
59 lstm_predictions_rescaled = scaler.inverse_transform(lstm_predictions)
60 y_test_rescaled = scaler.inverse_transform(y_test.reshape(-1, 1))
61
62 # Calculate RMSE for LSTM
63 lstm_rmse = math.sqrt(mean_squared_error(y_test_rescaled, lstm_predictions_rescaled))
64 print(f"LSTM RMSE: {lstm_rmse}")
65
66 # Step 5: Compare Performance
67 if arima_rmse < lstm_rmse:
68     print("ARIMA performs better with lower RMSE.")
69 else:
70     print("LSTM performs better with lower RMSE.")
```

Explanation of the Code

- 1. **Data Filtering** : The dataset is filtered to include only rows corresponding to adolescents (15–19) in the Americas. The HIV incidence column is extracted for analysis.
- 2. **Preprocessing** :
 - For **ARIMA**, the raw time series is split into training and testing sets.
 - For **LSTM**, the data is scaled and converted into sequences of fixed length (`seq_length`).
- 3. **Model Training** :
 - The **ARIMA** model is trained using the `statsmodels` library.
 - The **LSTM** model is built using Keras and trained on the scaled sequences.
- 4. **Evaluation** :
 - Both models are evaluated using RMSE on the test set.
 - The model with the lower RMSE is deemed better for this prediction task.

Expected Output

The script will output:

- RMSE for ARIMA.
- RMSE for LSTM.
- A statement indicating which model performs better.

Key Considerations

- **ARIMA** is simpler and works well for linear trends but may struggle with complex patterns.
- **LSTM** can capture non-linear relationships but requires more data and computational resources.

